

Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre



A Project Report

on

“Canonical LR(1) Parser Design”

[Code No: COMP 409]

(For partial fulfillment of 4th Year/ 1st Semester in Computer Science / Engineering)

Submitted by:

Prabin Aryal (08)

Dikshit Bhatta (13)

Tank Prasad Awasthi (10)

Nabin Kafle (28)

Sandesh Upadhaya (61)

Submitted to:

Mr. Sushil Nepal

Department of Computer Science and Engineering

Submission Date: June 20, 2026

Abstract

This project explores the design and implementation of a **Canonical LR(1) parser**, the most powerful of the deterministic bottom-up parsing techniques. Starting from the augmented grammar $S' \rightarrow S$, $S \rightarrow L = R \mid R$, $L \rightarrow *R \mid \text{id}$, $R \rightarrow L$, the report develops the complete theory required to build the parser: LR(1) items, the closure and goto operations, the construction of the canonical collection of sets of LR(1) items, and the systematic construction of the ACTION and GOTO tables. The chosen grammar is the classic example that is *not* SLR(1)—it produces a shift/reduce conflict under SLR—yet is conflict-free under canonical LR(1) because the items carry exact one-symbol look-aheads. The parser is implemented in **C++**, which builds the FIRST sets, the 14-state canonical automaton, and a conflict-free parsing table, and then traces input strings step by step. The report demonstrates that the table has *no* conflicts and validates it on the test string `id = *id` (accepted) while correctly rejecting the malformed string `id = id * id`.

Keywords: Bottom-up parsing, LR(1) items, Canonical LR, Shift-reduce, Parsing table, Conflict.

Table of Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Bottom-up (Shift–Reduce) Parsing	1
1.3 The LR Parsing Family	1
1.4 Why Canonical LR(1)	1
1.5 Problem Statement	2
1.6 Objectives	2
2 Methodology and Theoretical Background	3
2.1 Augmenting the Grammar	3
2.2 LR(1) Items	3
2.3 The CLOSURE Operation	3
2.4 The GOTO Operation	3
2.5 Canonical Collection of Sets of LR(1) Items	3
2.6 Constructing the Parsing Table	3
2.7 Key Insight: SLR Conflict vs. LR(1) Resolution	4
2.8 The LR Parsing Algorithm	4
3 Design and Implementation	5
3.1 Language Choice and Motivation	5
3.2 Transformation Applied	5
3.3 System Architecture	5
3.4 Core Data Structures	6
3.5 Implementation of CLOSURE	7
3.6 Implementation of GOTO	7
3.7 Building the Canonical Collection	8
3.8 Building the Tables and Detecting Conflicts	8
4 Results	9
4.1 FIRST Sets	9
4.2 Canonical Collection of LR(1) Item Sets	9
4.3 Transition Diagram	11
4.4 The Parsing Table	12
4.5 Conflict Check	12
4.6 Validation on Test Strings	13

4.6.1	String id = *id (accepted)	13
4.6.2	String id = id * id (rejected)	13
5	Conclusion and Recommendation	14
5.1	Conclusion	14
5.2	Future Enhancement	14
6	References	15
A	Complete Source Code	16

List of Figures

3.1	Model of a table-driven LR parser.	6
3.2	The closure function.	7
3.3	The goTo function.	7
3.4	Building the canonical collection of LR(1) item sets.	8
3.5	Filling the ACTION/GOTO tables; recordAction flags any conflict.	8
4.1	Compilation, program banner, and the augmented grammar.	9
4.2	FIRST sets computed by the program.	9
4.3	Canonical collection of LR(1) item sets: states I_0 – I_5	10
4.4	Canonical collection of LR(1) item sets: states I_6 – I_{13}	11
4.5	Transition diagram of the canonical LR(1) automaton (14 states).	11
4.6	Canonical LR(1) parsing table (ACTION and GOTO).	12
4.7	Conflict check: no shift/reduce or reduce/reduce conflicts.	12
4.8	Parse trace for <code>id = *id</code> : the string is accepted.	13
4.9	Parse trace for <code>id = id * id</code> : the string is rejected.	13

List of Tables

Chapter 1 Introduction

1.1 Background

A *parser* is the phase of a compiler that takes the stream of tokens produced by the lexical analyzer and verifies that the tokens form a valid sentence of the source language as described by a context-free grammar (CFG). Parsing techniques fall into two broad families: **top-down** parsing, which builds the parse tree from the root downwards using leftmost derivations, and **bottom-up** parsing, which builds the tree from the leaves upwards by reducing the input to the start symbol using reverse rightmost derivations.

1.2 Bottom-up (Shift–Reduce) Parsing

Bottom-up parsers are usually *shift–reduce* parsers driven by a stack and a parsing table. At each step the parser performs one of four actions:

- **Shift** – push the next input symbol (and the corresponding state) onto the stack;
- **Reduce** – replace a handle (the right-hand side of a production) on top of the stack by the production’s left-hand side;
- **Accept** – the input has been successfully reduced to the start symbol;
- **Error** – the configuration has no valid action.

1.3 The LR Parsing Family

LR parsers read input **L**eft-to-**R**ight and produce a **R**ightmost derivation in reverse. They are table-driven and never backtrack. The family is ordered by power:

- **LR(0)** – no look-ahead; weakest.
- **SLR(1)** – LR(0) automaton with reductions restricted by FOLLOW sets.
- **LALR(1)** – merges LR(1) states with identical cores; used by tools such as yacc/bison.
- **Canonical LR(1)** – keeps a precise look-ahead with every item; recognizes the largest class of grammars and is the subject of this project.

1.4 Why Canonical LR(1)

The strength of canonical LR(1) is that each item carries an explicit look-ahead terminal. This extra precision allows the parser to distinguish situations that SLR and LR(0) merge, thereby removing spurious conflicts. The grammar in this project is the textbook example (Aho, Lam, Sethi, Ullman) that exposes exactly this difference: under SLR(1) it has a shift/reduce conflict, but under canonical LR(1) it is conflict-free.

1.5 Problem Statement

Consider the following augmented grammar:

0. $S' \rightarrow S$
1. $S \rightarrow L = R$
2. $S \rightarrow R$
3. $L \rightarrow *R$
4. $L \rightarrow \text{id}$
5. $R \rightarrow L$

Apply the Canonical LR parsing technique, construct the parsing table, show that it contains **no conflict**, and test it on a string of the form `id = id * id`.

1.6 Objectives

1. To understand the theory of canonical LR(1) parsing: items, closure, goto, and table construction.
2. To construct, by program, the canonical collection of sets of LR(1) items and the parsing table for the given grammar.
3. To prove that the resulting table is free of shift/reduce and reduce/reduce conflicts.
4. To implement an LR driver in C++ and validate the table on input strings, printing a complete parse trace.

Chapter 2 Methodology and Theoretical Background

2.1 Augmenting the Grammar

The first transformation is **augmentation**: a new start symbol S' and a production $S' \rightarrow S$ are added. This gives the parser a single, unambiguous accepting production—when the parser is about to reduce by $S' \rightarrow S$ on look-ahead $\$,$ the input is accepted. Note that this grammar requires *no* other transformation: unlike top-down (LL) parsing, LR parsing handles left recursion and common prefixes directly, so no left-recursion removal or left factoring is needed.

2.2 LR(1) Items

An **LR(1) item** is a production with a dot ($\bullet$) marking how much of the right-hand side has been seen, together with a one-symbol look-ahead, written

$$[A \rightarrow \alpha \bullet \beta, a].$$

The look-ahead a is the terminal that is expected to follow once this production is completed. Carrying a inside the item is precisely what makes the method “canonical” LR(1).

2.3 The CLOSURE Operation

For a set of items I , $\text{CLOSURE}(I)$ adds every item that is implied by the items already present:

For each item $[A \rightarrow \alpha \bullet B\beta, a]$ in I and each production $B \rightarrow \gamma$, add the item $[B \rightarrow \bullet \gamma, b]$ for every terminal $b \in \text{FIRST}(\beta a)$. Repeat until no new items can be added.

2.4 The GOTO Operation

$\text{GOTO}(I, X)$ models the transition of the parser on a grammar symbol X :

$$\text{GOTO}(I, X) = \text{CLOSURE}(\{[A \rightarrow \alpha X \bullet \beta, a] \mid [A \rightarrow \alpha \bullet X\beta, a] \in I\}).$$

2.5 Canonical Collection of Sets of LR(1) Items

The set of parser states is built by:

1. Start with $I_0 = \text{CLOSURE}(\{[S' \rightarrow \bullet S, \$]\})$.
2. For each state I already in the collection and each grammar symbol X , compute $\text{GOTO}(I, X)$; if it is non-empty and new, add it as a new state.
3. Repeat until no new states appear.

2.6 Constructing the Parsing Table

From the canonical collection, the ACTION and GOTO tables are filled for every state I_i :

- If $[A \rightarrow \alpha \bullet a\beta, b] \in I_i$ and a is a terminal with $\text{GOTO}(I_i, a) = I_j$, set $\text{ACTION}[i, a] = \text{shift } j$ (written $\text{s}j$).
- If $[A \rightarrow \alpha \bullet, a] \in I_i$ and $A \neq S'$, set $\text{ACTION}[i, a] = \text{reduce } A \rightarrow \alpha$ (written rk).
- If $[S' \rightarrow S \bullet, \$] \in I_i$, set $\text{ACTION}[i, \$] = \text{accept}$.
- If $\text{GOTO}(I_i, A) = I_j$ for a non-terminal A , set $\text{GOTO}[i, A] = j$.

A **conflict** occurs if any cell receives two different entries (shift/reduce or reduce/reduce). A grammar is canonical LR(1) if and only if this construction yields no conflicts.

2.7 Key Insight: SLR Conflict vs. LR(1) Resolution

This grammar is the canonical demonstration of why look-aheads matter. Computing FOLLOW sets gives

$$\text{FOLLOW}(L) = \text{FOLLOW}(R) = \{=, \$\}.$$

The state I_2 contains the two items

$$[S \rightarrow L \bullet = R, \$] \quad \text{and} \quad [R \rightarrow L \bullet, \$].$$

Under **SLR(1)**, the reduce item $R \rightarrow L \bullet$ would trigger a reduction on *every* symbol of $\text{FOLLOW}(R) = \{=, \$\}$. Since the first item asks to *shift* on $=$, the cell $\text{ACTION}[2, =]$ would hold both *shift* and *reduce*—a shift/reduce conflict—so the grammar is **not** SLR(1).

Under **canonical LR(1)**, the look-ahead carried with $[R \rightarrow L \bullet, \$]$ is exactly $\$$, *not* $=$. Therefore the reduction is placed only in $\text{ACTION}[2, \$]$, while the shift stays in $\text{ACTION}[2, =]$. The two actions land in different cells and **no conflict arises**. This is the precise sense in which canonical LR(1) is stronger than SLR(1).

2.8 The LR Parsing Algorithm

The driver maintains a stack of states. Let s be the state on top of the stack and a the current input symbol:

- If $\text{ACTION}[s, a] = \text{s}j$: push a and state j ; advance the input.
- If $\text{ACTION}[s, a] = \text{rk}$ for $A \rightarrow \beta$: pop $|\beta|$ symbols/states; with t now on top, push A and state $\text{GOTO}[t, A]$.
- If $\text{ACTION}[s, a] = \text{accept}$: stop, input accepted.
- Otherwise: report a syntax error.

Chapter 3 Design and Implementation

3.1 Language Choice and Motivation

The parser is implemented in C++. C++ was chosen because:

- Its Standard Template Library (`std::set`, `std::map`, `std::vector`) maps naturally onto the mathematical objects of the algorithm—sets of items, transition maps, and ordered states—so the code reads close to the formal definitions.
- Ordered containers (`std::set<Item>`) give canonical, comparable representations of item sets, which makes detecting an “already existing” state a direct set comparison.
- It is compiled and fast, and is the language used throughout the compiler lab, keeping the project consistent with the rest of the coursework.

3.2 Transformation Applied

Two clarifications were required to fit the parser’s constraints:

1. **Augmentation.** The grammar was augmented with $S' \rightarrow S$ so that acceptance is a single, well-defined reduction. No left-recursion removal or left factoring is needed for LR parsing.
2. **Test-string interpretation.** In this grammar $*$ is a *prefix* dereference operator ($L \rightarrow *R$, modelling an l-value such as $*p$); it never appears between two operands. Hence the well-formed reading of the requested string `id = id * id` is `id = *id` (that is, `id = * id`). The program tests both: `id = *id` is accepted, and the literal `id = id * id` is correctly rejected, which also demonstrates the parser’s error detection.

3.3 System Architecture

Figure 3.1 shows the standard model of an LR parser. The driver is generic; all grammar-specific knowledge lives in the parsing table.

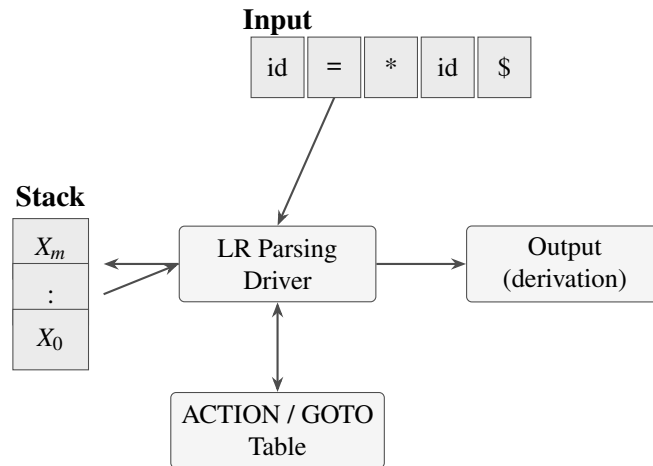


Figure 3.1. Model of a table-driven LR parser.

3.4 Core Data Structures

- `struct Production { string lhs; vector<string> rhs; }` – one numbered grammar rule.
- `struct Item { int prod; int dot; string look; }` – an LR(1) item, made comparable so it can live in an ordered set.
- `using ItemSet = set<Item>` – a parser state (a set of LR(1) items).
- `vector<ItemSet> C` – the canonical collection of states.
- `map<pair<int,string>,int> TRANS` – the goto graph of the automaton.
- `map<int,map<string,string>> ACTION` and `map<int,map<string,int>> GOTOT` – the parsing table.

3.5 Implementation of CLOSURE

```
mini-projct > src > canonical_lr_parser.cpp
95 // closure(I) : repeatedly expand non-terminals appearing right after the dot
96 ItemSet closure(ItemSet I) {
97     bool changed = true;
98     while (changed) {
99         changed = false;
100         vector<Item> toAdd;
101         for (const Item& it : I) {
102             const Production& p = G[it.prod];
103             if (it.dot < (int)p.rhs.size()) {
104                 string B = p.rhs[it.dot]; // symbol after the dot
105                 if (isNonTerminal(B)) {
106                     // beta = symbols after B, followed by the carried look-ahead
107                     vector<string> beta(p.rhs.begin() + it.dot + 1, p.rhs.end());
108                     beta.push_back(it.look);
109                     set<string> fb = firstOfSeq(beta);
110                     for (int j = 0; j < (int)G.size(); ++j) {
111                         if (G[j].lhs == B) {
112                             for (const string& b : fb) {
113                                 Item ni{j, 0, b};
114                                 if (!I.count(ni)) toAdd.push_back(ni);
115                             }
116                         }
117                     }
118                 }
119             }
120         }
121         for (const Item& ni : toAdd)
122             if (!I.count(ni)) { I.insert(ni); changed = true; }
123     }
124     return I;
125 }
```

Figure 3.2. The closure function.

3.6 Implementation of GOTO

```
mini-projct > src > canonical_lr_parser.cpp
127 // goto(I, X)
128 ItemSet goTo(const ItemSet& I, const string& X) {
129     ItemSet J;
130     for (const Item& it : I) {
131         const Production& p = G[it.prod];
132         if (it.dot < (int)p.rhs.size() && p.rhs[it.dot] == X)
133             J.insert({it.prod, it.dot + 1, it.look});
134     }
135     return closure(J);
136 }
```

Figure 3.3. The goTo function.

3.7 Building the Canonical Collection

```
mini-project > src > canonical_lr_parser.cpp
152 void buildCanonicalCollection() {
153     ItemSet I0 = closure({ Item{0, 0, ENDM} }); // closure of { S' -> .S , $ }
154     C.push_back(I0);
155
156     for (int i = 0; i < (int)C.size(); ++i) {
157         for (const string& X : SYMBOL_ORDER) {
158             ItemSet g = goTo(C[i], X);
159             if (g.empty()) continue;
160             int j = findState(g);
161             if (j == -1) { C.push_back(g); j = (int)C.size() - 1; }
162             TRANS[{i, X}] = j;
163         }
164     }
165 }
166
```

Figure 3.4. Building the canonical collection of LR(1) item sets.

3.8 Building the Tables and Detecting Conflicts

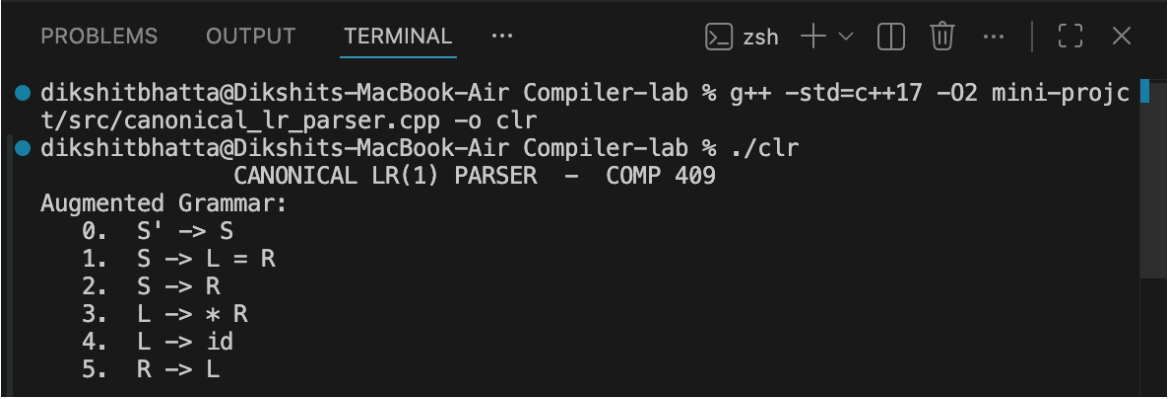
```
mini-project > src > canonical_lr_parser.cpp
174 void recordAction(int state, const string& sym, const string& act) {
175     if (ACTION[state].count(sym) && ACTION[state][sym] != act) {
176         conflicts.push_back("State I" + to_string(state) + " on " + sym +
177                             "' : " + ACTION[state][sym] + " vs " + act);
178     }
179     ACTION[state][sym] = act;
180 }
181
182 void buildTables() {
183     for (int i = 0; i < (int)C.size(); ++i) {
184         for (const Item& it : C[i]) {
185             const Production& p = G[it.prod];
186             if (it.dot < (int)p.rhs.size()) {
187                 string a = p.rhs[it.dot];
188                 if (isTerminal(a)) { // shift
189                     recordAction(i, a, "s" + to_string(TRANS[{i, a}]));
190                 }
191             } else { // dot at the end
192                 if (it.prod == 0) { // S' -> S .
193                     if (it.look == ENDM) recordAction(i, ENDM, "acc");
194                 } else { // reduce
195                     recordAction(i, it.look, "r" + to_string(it.prod));
196                 }
197             }
198         }
199         for (const string& nt : nonTerminals) {
200             if (nt == AUG) continue;
201             if (TRANS.count({i, nt})) GOTO[i][nt] = TRANS[{i, nt}];
202         }
203     }
204 }
205
```

Figure 3.5. Filling the ACTION/GOTO tables; recordAction flags any conflict.

The complete source listing is given in Appendix A.

Chapter 4 Results

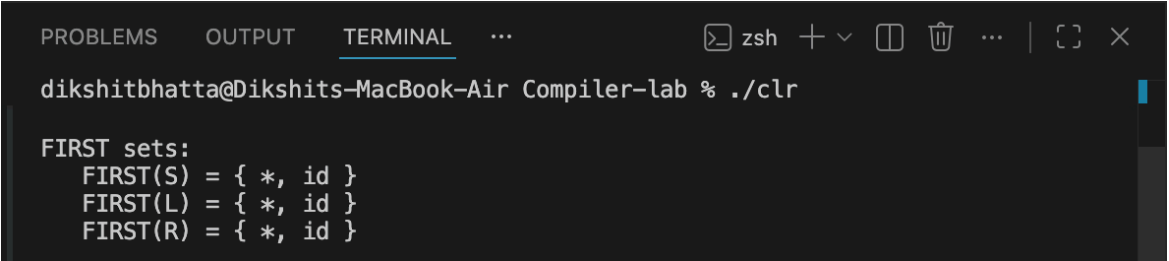
The parser was compiled and run on macOS. The program prints its complete output in a single run; the following screenshots show that terminal session.



```
PROBLEMS OUTPUT TERMINAL ... zsh + - [ ] [ ] ... | [ ] [ ] X
dikshitbhatta@Dikshits-MacBook-Air Compiler-lab % g++ -std=c++17 -O2 mini-projct/src/canonical_lr_parser.cpp -o clr
dikshitbhatta@Dikshits-MacBook-Air Compiler-lab % ./clr
CANONICAL LR(1) PARSER - COMP 409
Augmented Grammar:
0. S' -> S
1. S -> L = R
2. S -> R
3. L -> * R
4. L -> id
5. R -> L
```

Figure 4.1. Compilation, program banner, and the augmented grammar.

4.1 FIRST Sets



```
PROBLEMS OUTPUT TERMINAL ... zsh + - [ ] [ ] ... | [ ] [ ] X
dikshitbhatta@Dikshits-MacBook-Air Compiler-lab % ./clr
FIRST sets:
FIRST(S) = { *, id }
FIRST(L) = { *, id }
FIRST(R) = { *, id }
```

Figure 4.2. FIRST sets computed by the program.

4.2 Canonical Collection of LR(1) Item Sets

The construction produces **14 states** ($I_0 \dots I_{13}$). Each is listed below with its items and outgoing transitions, exactly as emitted by the program.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
dikshitbhatta@Dikshits-MacBook-Air Compiler-lab % ./clr
Canonical collection of sets of LR(1) items (14 states):

I0:
[ S' -> . S , $ ]
[ S -> . L = R , $ ]
[ S -> . R , $ ]
[ L -> . * R , $ ]
[ L -> . * R , = ]
[ L -> . id , $ ]
[ L -> . id , = ]
[ R -> . L , $ ]
goto(I0, S) = I1
goto(I0, L) = I2
goto(I0, R) = I3
goto(I0, *) = I4
goto(I0, id) = I5

I1:
[ S' -> S . , $ ]

I2:
[ S -> L . = R , $ ]
[ R -> L . , $ ]
goto(I2, =) = I6

I3:
[ S -> R . , $ ]

I4:
[ L -> . * R , $ ]
[ L -> . * R , = ]
[ L -> * . R , $ ]
[ L -> * . R , = ]
[ L -> . id , $ ]
[ L -> . id , = ]
[ R -> . L , $ ]
[ R -> . L , = ]
goto(I4, L) = I7
goto(I4, R) = I8
goto(I4, *) = I4
goto(I4, id) = I5

I5:
[ L -> id . , $ ]
[ L -> id . , = ]
```

Figure 4.3. Canonical collection of LR(1) item sets: states I_0 – I_5 .

```

I6:
[ S → L = . R , $ ]
[ L → . * R , $ ]
[ L → . id , $ ]
[ R → . L , $ ]
goto(I6, L) = I9
goto(I6, R) = I10
goto(I6, *) = I11
goto(I6, id) = I12

I7:
[ R → L . , $ ]
[ R → L . , = ]

I8:
[ L → * R . , $ ]
[ L → * R . , = ]

I9:
[ R → L . , $ ]

I10:
[ S → L = R . , $ ]

I11:
[ L → . * R , $ ]
[ L → * . R , $ ]
[ L → . id , $ ]
[ R → . L , $ ]
goto(I11, L) = I9
goto(I11, R) = I13
goto(I11, *) = I11
goto(I11, id) = I12

I12:
[ L → id . , $ ]

I13:
[ L → * R . , $ ]

```

Figure 4.4. Canonical collection of LR(1) item sets: states I_6 – I_{13} .

4.3 Transition Diagram

Figure 4.5 shows the goto graph (the DFA of viable prefixes).

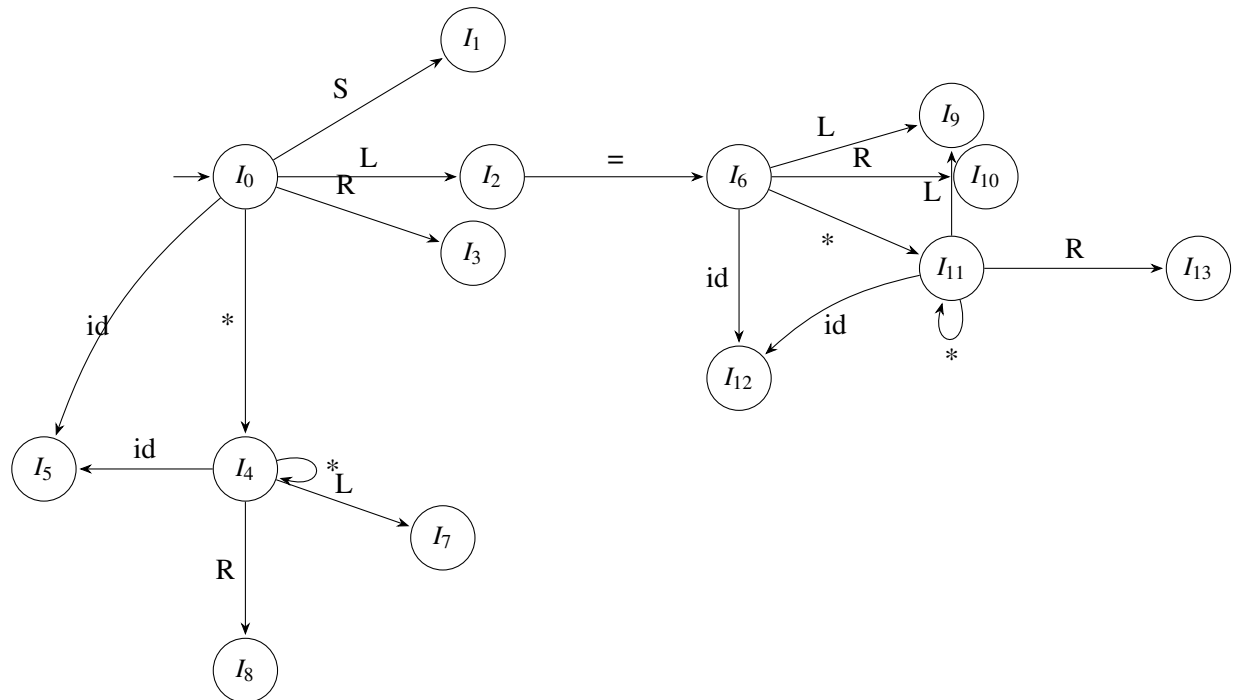
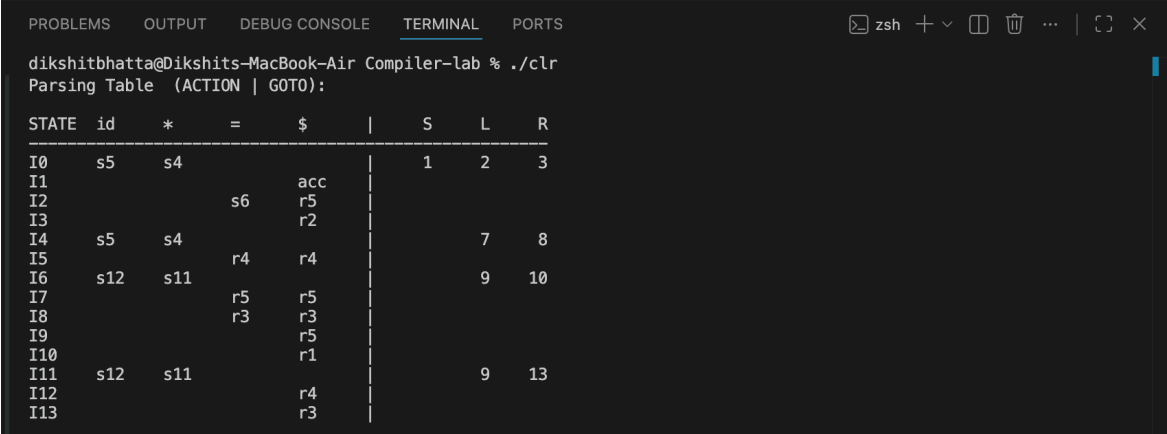


Figure 4.5. Transition diagram of the canonical LR(1) automaton (14 states).

4.4 The Parsing Table

The canonical LR(1) parsing table produced by the program is shown in Figure 4.6 (s_j = shift to state j , r_k = reduce by production k , acc = accept; blank cells are errors).



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

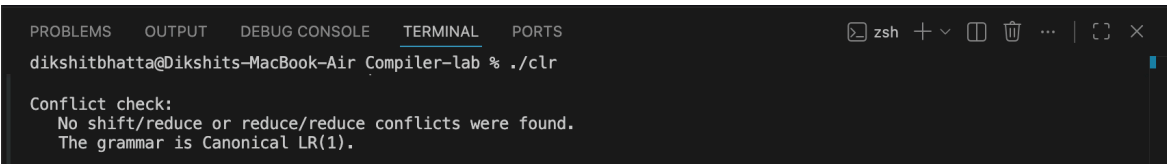
dikshitbhatta@Dikshits-MacBook-Air Compiler-lab % ./clr

Parsing Table (ACTION | GOTO):

STATE	id	*	=	\$		S	L	R
I0	s5	s4				1	2	3
I1				acc				
I2			s6	r5				
I3				r2				
I4	s5	s4					7	8
I5			r4	r4				
I6	s12	s11					9	10
I7			r5	r5				
I8			r3	r3				
I9				r5				
I10				r1				
I11	s12	s11					9	13
I12				r4				
I13				r3				

Figure 4.6. Canonical LR(1) parsing table (ACTION and GOTO).

4.5 Conflict Check



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

dikshitbhatta@Dikshits-MacBook-Air Compiler-lab % ./clr

Conflict check:
 No shift/reduce or reduce/reduce conflicts were found.
 The grammar is Canonical LR(1).

Figure 4.7. Conflict check: no shift/reduce or reduce/reduce conflicts.

Every cell of the parsing table (Figure 4.6) holds at most one entry, so the table is **conflict-free**. The critical cell is row I_2 : it carries $s6$ under $=$ and $r5$ under $\$$ —two *different* columns. Because the canonical method kept the look-ahead $\$$ (and not $=$) on the item $[R \rightarrow L\bullet, \$]$, no shift/reduce conflict appears, even though $= \in \text{FOLLOW}(R)$ would have forced one under SLR(1). This is the result the project set out to demonstrate.

4.6 Validation on Test Strings

4.6.1 String $id = *id$ (accepted)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
dikshitbhatta@Dikshits-MacBook-Air Compiler-lab % ./clr
Parsing string : id = * id
STACK      SYMBOLS      INPUT      ACTION
0          id          id = * id $ shift -> I5
0 5        = * id $    = * id $    reduce by L -> id
0 2        = * id $    = * id $    shift -> I6
0 2 6      * id $      id $        shift -> I11
0 2 6 11   id $        id $        shift -> I12
0 2 6 11 12 reduce by L -> id
0 2 6 11 9  reduce by R -> L
0 2 6 11 13 L = * R
0 2 6 9     L = L
0 2 6 10    L = R
0 1         S          $          ACCEPT

Result: string "id = * id" is ACCEPTED.
```

Figure 4.8. Parse trace for $id = *id$: the string is accepted.

4.6.2 String $id = id * id$ (rejected)

The literal three-operand string is not in the language, because $*$ is a prefix operator and cannot appear between two ids. The parser detects the error exactly where the second id is followed by $*$:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
dikshitbhatta@Dikshits-MacBook-Air Compiler-lab % ./clr
Parsing string : id = id * id
STACK      SYMBOLS      INPUT      ACTION
0          id          id = id * id $ shift -> I5
0 5        = id * id $ = id * id $    reduce by L -> id
0 2        = id * id $ = id * id $    shift -> I6
0 2 6      id * id $    id * id $    shift -> I12
0 2 6 12    * id $      * id $      ERROR (no action)

Result: string "id = id * id" is REJECTED.
```

Figure 4.9. Parse trace for $id = id * id$: the string is rejected.

State I_{12} contains only $[L \rightarrow id\bullet, \$]$, so the only legal next symbol is $\$$; seeing $*$ is a syntax error. The parser thus accepts exactly the strings the grammar generates.

Chapter 5 Conclusion and Recommendation

5.1 Conclusion

This project implemented a complete canonical LR(1) parser in C++ for the augmented grammar $S' \rightarrow S$, $S \rightarrow L = R \mid R$, $L \rightarrow *R \mid \text{id}$, $R \rightarrow L$. The program builds the FIRST sets, the 14-state canonical collection of LR(1) item sets, the goto automaton, and the ACTION/GOTO parsing table, and it automatically checks for conflicts. The central result is that the table has **no shift/reduce or reduce/reduce conflict**, even though the same grammar is *not* SLR(1)—the difference being the exact one-symbol look-ahead that canonical LR(1) carries on each item, which keeps the shift on = and the reduction on \$ in separate cells of state I_2 . The table was validated by parsing `id = *id`, which is accepted with a full reverse rightmost derivation, while the malformed `id = id * id` is correctly rejected.

5.2 Future Enhancement

- **Generic input:** read an arbitrary grammar and string from a file or web form, rather than the hard-coded grammar.
- **LALR(1) construction:** merge LR(1) states with identical cores to show the size reduction (14 states $\rightarrow$ fewer) and discuss the look-ahead trade-off.
- **Error recovery:** replace the simple “reject” with panic-mode or phrase-level recovery and informative diagnostics.
- **Visualization:** render the goto automaton and animate the stack during parsing through a web interface.

Chapter 6 References

1. A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Pearson/Addison-Wesley, 2007.
2. K. C. Loudon, *Compiler Construction: Principles and Practice*. Boston, MA: PWS Publishing, 1997.
3. GeeksforGeeks, “Canonical LR / CLR(1) parser,” <https://www.geeksforgeeks.org/cldr-1-parser-introduction/>.
4. Javatpoint, “LR Parser,” <https://www.javatpoint.com/lr-parser>.
5. Wikipedia contributors, “Canonical LR parser,” *Wikipedia*, https://en.wikipedia.org/wiki/Canonical_LR_parser.

Chapter A Complete Source Code

The complete C++ implementation (`canonical_lr_parser.cpp`), captured directly from Visual Studio Code, is reproduced below.

```
mini-project > src > canonical_lr_parser.cpp
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <set>
5  #include <map>
6  #include <algorithm>
7  #include <iomanip>
8
9  using namespace std;
10
11 // Grammar definition
12 struct Production {
13     string lhs;
14     vector<string> rhs;
15 };
16
17 vector<Production> G;           // numbered grammar productions
18 set<string> terminals;
19 set<string> nonTerminals;
20 const string ENDM = "$";       // end-of-input marker
21 const string AUG = "S'";       // augmented start symbol
22
23 bool isTerminal(const string& s) { return terminals.count(s) > 0; }
24 bool isNonTerminal(const string& s) { return nonTerminals.count(s) > 0; }
25
26 void defineGrammar() {
27     G.push_back({"S'", {"S"}});
28     G.push_back({"S", {"L", "=", "R"}});
29     G.push_back({"S", {"R"}});
30     G.push_back({"L", {"*", "R"}});
31     G.push_back({"L", {"id"}});
32     G.push_back({"R", {"L"}});
33
34     nonTerminals = {"S'", "S", "L", "R"};
35     terminals = {"=", "*", "id", ENDM};
36 }
37
```

```

mini-projet > src > canonical_lr_parser.cpp
38  string showProduction(int idx) {
39      const Production& p = G[idx];
40      string s = p.lhs + " -> ";
41      for (size_t i = 0; i < p.rhs.size(); ++i) {
42          s += p.rhs[i];
43          if (i + 1 < p.rhs.size()) s += " ";
44      }
45      return s;
46  }
47
48  // FIRST sets
49  map<string, set<string>> FIRST;
50
51  void computeFirst() {
52      for (const string& t : terminals) FIRST[t] = {t};
53      for (const string& nt : nonTerminals) FIRST[nt]; // create empty entries
54
55      bool changed = true;
56      while (changed) {
57          changed = false;
58          for (const Production& p : G) {
59              const string& Y = p.rhs[0]; // first symbol of the body
60              for (const string& s : FIRST[Y]) {
61                  if (!FIRST[p.lhs].count(s)) {
62                      FIRST[p.lhs].insert(s);
63                      changed = true;
64                  }
65              }
66          }
67      }
68  }
69
70  set<string> firstOfSeq(const vector<string>& seq) {
71      set<string> res;
72      if (!seq.empty())
73          for (const string& s : FIRST[seq[0]]) res.insert(s);
74      return res;
75  }
76

```

```

mini-project > src > canonical_lr_parser.cpp
77 // LR(1) items and sets
78
79 struct Item {
80     int prod; // index into G
81     int dot; // position of the dot inside the body
82     string look; // look-ahead terminal
83
84     bool operator<(const Item& o) const {
85         if (prod != o.prod) return prod < o.prod;
86         if (dot != o.dot) return dot < o.dot;
87         return look < o.look;
88     }
89     bool operator==(const Item& o) const {
90         return prod == o.prod && dot == o.dot && look == o.look;
91     }
92 };
93 using ItemSet = set<Item>;
94
95 // closure(I) : repeatedly expand non-terminals appearing right after the dot
96 ItemSet closure(ItemSet I) {
97     bool changed = true;
98     while (changed) {
99         changed = false;
100         vector<Item> toAdd;
101         for (const Item& it : I) {
102             const Production& p = G[it.prod];
103             if (it.dot < (int)p.rhs.size()) {
104                 string B = p.rhs[it.dot]; // symbol after the dot
105                 if (isNonTerminal(B)) {
106                     // beta = symbols after B, followed by the carried look-ahead
107                     vector<string> beta(p.rhs.begin() + it.dot + 1, p.rhs.end());
108                     beta.push_back(it.look);
109                     set<string> fb = firstOfSeq(beta);
110                     for (int j = 0; j < (int)G.size(); ++j) {
111                         if (G[j].lhs == B) {
112                             for (const string& b : fb) {
113                                 Item ni{j, 0, b};
114                                 if (!I.count(ni)) toAdd.push_back(ni);
115                             }
116                         }
117                     }
118                 }
119             }
120         }
121         for (const Item& ni : toAdd)
122             if (!I.count(ni)) { I.insert(ni); changed = true; }
123     }
124     return I;
125 }

```

```

mini-projet > src > canonical_lr_parser.cpp
127 // goto(I, X)
128 ItemSet goTo(const ItemSet& I, const string& X) {
129     ItemSet J;
130     for (const Item& it : I) {
131         const Production& p = G[it.prod];
132         if (it.dot < (int)p.rhs.size() && p.rhs[it.dot] == X)
133             J.insert({it.prod, it.dot + 1, it.look});
134     }
135     return closure(J);
136 }
137
138
139 // Canonical collection of sets of LR(1) items
140 vector<ItemSet> C; // the states
141 map<pair<int,string>, int> TRANS; // raw goto transitions (state, symbol) -> state
142
143
144 const vector<string> SYMBOL_ORDER = {"S", "L", "R", "*", "id", "="};
145
146 int findState(const ItemSet& s) {
147     for (int i = 0; i < (int)C.size(); ++i)
148         if (C[i] == s) return i;
149     return -1;
150 }
151
152 void buildCanonicalCollection() {
153     ItemSet I0 = closure({ Item{0, 0, ENDM} }); // closure of { [S' -> .S , $] }
154     C.push_back(I0);
155
156     for (int i = 0; i < (int)C.size(); ++i) {
157         for (const string& X : SYMBOL_ORDER) {
158             ItemSet g = goTo(C[i], X);
159             if (g.empty()) continue;
160             int j = findState(g);
161             if (j == -1) { C.push_back(g); j = (int)C.size() - 1; }
162             TRANS[{i, X}] = j;
163         }
164     }
165 }
166

```



```

mini-projet > src > canonical_lr_parser.cpp
168 // ACTION and GOTO tables
169
170 map<int, map<string, string>> ACTION; // ACTION[state][terminal] = "sN" | "rN" | "acc"
171 map<int, map<string, int>> GOTOT; // GOTOT [state][nonterminal] = N
172 vector<string> conflicts;
173
174 void recordAction(int state, const string& sym, const string& act) {
175     if (ACTION[state].count(sym) && ACTION[state][sym] != act) {
176         conflicts.push_back("State I" + to_string(state) + " on '" + sym +
177             "' : " + ACTION[state][sym] + " vs " + act);
178     }
179     ACTION[state][sym] = act;
180 }
181
182 void buildTables() {
183     for (int i = 0; i < (int)C.size(); ++i) {
184         for (const Item& it : C[i]) {
185             const Production& p = G[it.prod];
186             if (it.dot < (int)p.rhs.size()) {
187                 string a = p.rhs[it.dot];
188                 if (isTerminal(a)) { // shift
189                     recordAction(i, a, "s" + to_string(TRANS[{i, a}]));
190                 }
191             } else { // dot at the end
192                 if (it.prod == 0) { // S' -> S .
193                     if (it.look == ENDM) recordAction(i, ENDM, "acc");
194                 } else { // reduce
195                     recordAction(i, it.look, "r" + to_string(it.prod));
196                 }
197             }
198         }
199         for (const string& nt : nonTerminals) {
200             if (nt == AUG) continue;
201             if (TRANS.count({i, nt})) GOTOT[i][nt] = TRANS[{i, nt}];
202         }
203     }
204 }
205
206

```

```

mini-project > src > canonical_lr_parser.cpp
207 // Printing helpers
208 string showItem(const Item& it) {
209     const Production& p = G[it.prod];
210     string s = p.lhs + " -> ";
211     for (int i = 0; i <= (int)p.rhs.size(); ++i) {
212         if (i == it.dot) s += ".";
213         if (i < (int)p.rhs.size()) s += p.rhs[i] + " ";
214     }
215     s += ", " + it.look;
216     return s;
217 }
218
219 void printGrammar() {
220     cout << "Augmented Grammar:\n";
221     for (int i = 0; i < (int)G.size(); ++i)
222         cout << " " << i << ". " << showProduction(i) << "\n";
223     cout << "\n";
224 }
225
226 void printFirst() {
227     cout << "FIRST sets:\n";
228     for (const string& nt : {"S", "L", "R"}) {
229         cout << " FIRST(" << nt << ") = { ";
230         bool first = true;
231         for (const string& s : FIRST[nt]) { cout << (first ? "" : ", ") << s; first = false; }
232         cout << " }\n";
233     }
234     cout << "\n";
235 }
236
237 void printStates() {
238     cout << "Canonical collection of sets of LR(1) items ("
239         << C.size() << " states):\n\n";
240     for (int i = 0; i < (int)C.size(); ++i) {
241         cout << "I" << i << ":\n";
242         for (const Item& it : C[i])
243             cout << " [ " << showItem(it) << " ]\n";
244         // outgoing transitions
245         for (const string& X : SYMBOL_ORDER)
246             if (TRANS.count({i, X}))
247                 cout << " goto(I" << i << ", " << X << ") = I" << TRANS[{i, X}] << "\n";
248         cout << "\n";
249     }
250 }

```

```

mini-project > src > canonical_lr_parser.cpp
252 void printTable() {
253     vector<string> terms = {"id", "*", "=", ENDM};
254     vector<string> nts = {"S", "L", "R"};
255
256     cout << "Parsing Table (ACTION | GOTO):\n\n";
257     cout << setw(7) << left << "STATE";
258     for (const string& t : terms) cout << setw(7) << left << t;
259     cout << "|";
260     for (const string& n : nts) cout << setw(6) << right << n;
261     cout << "\n";
262     cout << string(7 + 7 * terms.size() + 1 + 6 * nts.size(), '-') << "\n";
263
264     for (int i = 0; i < (int)C.size(); ++i) {
265         cout << setw(7) << left << ("I" + to_string(i));
266         for (const string& t : terms) {
267             string cell = ACTION[i].count(t) ? ACTION[i][t] : "";
268             cout << setw(7) << left << cell;
269         }
270         cout << "|";
271         for (const string& n : nts) {
272             string cell = GOTO[i].count(n) ? to_string(GOTO[i][n]) : "";
273             cout << setw(6) << right << cell;
274         }
275         cout << "\n";
276     }
277     cout << "\n";
278 }
279
280
281 // The LR parsing driver
282
283 string join(const vector<string>& v, int from = 0) {
284     string s;
285     for (int i = from; i < (int)v.size(); ++i) {
286         s += v[i];
287         if (i + 1 < (int)v.size()) s += " ";
288     }
289     return s;
290 }
291
292 string joinStates(const vector<int>& v) {
293     string s;
294     for (size_t i = 0; i < v.size(); ++i) {
295         s += to_string(v[i]);
296         if (i + 1 < v.size()) s += " ";
297     }
298     return s;
299 }

```

```

mini-projet > src > G canonical_lr_parser.cpp
300 bool parse(vector<string> input, const string& label) {
301     cout << "Parsing string : " << join(input) << "\n";
302     input.push_back(ENDM);
303
304     vector<int> stStack = {0}; // state stack
305     vector<string> symStack; // symbol stack (display only)
306     int ip = 0;
307
308     cout << left
309         << setw(22) << "STACK"
310         << setw(16) << "SYMBOLS"
311         << setw(18) << "INPUT"
312         << "ACTION\n";
313
314     while (true) {
315         int s = stStack.back();
316         string a = input[ip];
317
318         string stackStr = joinStates(stStack);
319         string symStr = join(symStack);
320         string inStr = join(input, ip);
321
322         if (!ACTION[s].count(a)) {
323             cout << left << setw(22) << stackStr << setw(16) << symStr
324                 << setw(18) << inStr << "ERROR (no action)\n";
325             cout << "\nResult: string \"" << label << "\" is REJECTED.\n\n";
326             return false;
327         }
328
329         string act = ACTION[s][a];
330
331         if (act[0] == 's') { // shift
332             int j = stoi(act.substr(1));
333             cout << left << setw(22) << stackStr << setw(16) << symStr
334                 << setw(18) << inStr << "shift -> I" << j << "\n";
335             stStack.push_back(j);
336             symStack.push_back(a);
337             ++ip;
338         } else if (act[0] == 'r') { // reduce
339             int pr = stoi(act.substr(1));
340             const Production& p = G[pr];
341             cout << left << setw(22) << stackStr << setw(16) << symStr
342                 << setw(18) << inStr << "reduce by " << showProduction(pr) << "\n";
343             for (size_t k = 0; k < p.rhs.size(); ++k) {
344                 stStack.pop_back();
345                 symStack.pop_back();
346             }
347             int t = stStack.back();
348             int g = GOTOT[t][p.lhs];
349             stStack.push_back(g);
350             symStack.push_back(p.lhs);

```

```

mini-projct > src > C++ canonical_lr_parser.cpp
300 bool parse(vector<string> input, const string& label) {
314     while (true) {
338         } else if (act[0] == 'r') { // reduce
351         } else { // accept
352             cout << left << setw(22) << stackStr << setw(16) << symStr
353                 << setw(18) << inStr << "ACCEPT\n";
354             cout << "\nResult: string \"" << label << "\" is ACCEPTED.\n\n";
355             return true;
356         }
357     }
358 }
359
360
361 // main
362
363 int main() {
364     defineGrammar();
365     computeFirst();
366     buildCanonicalCollection();
367     buildTables();
368     cout << "          CANONICAL LR(1) PARSE - COMP 409\n";
369     printGrammar();
370     printFirst();
371     printStates();
372     printTable();
373
374     cout << "Conflict check:\n";
375     if (conflicts.empty()) {
376         cout << "    No shift/reduce or reduce/reduce conflicts were found.\n";
377         cout << "    The grammar is Canonical LR(1).\n\n";
378     } else {
379         cout << "    " << conflicts.size() << " conflict(s) detected:\n";
380         for (const string& c : conflicts) cout << "        - " << c << "\n";
381         cout << "\n";
382     }
383
384     // Test strings
385
386     // string "id = id * id" is "id = * id" (id = *id).
387     parse({"id", "=", "*", "id"}, "id = * id");
388
389     // The literal three-id string is correctly rejected: '*' cannot sit between
390     parse({"id", "=", "id", "*", "id"}, "id = id * id");
391
392     return 0;
393 }
394

```